H inf Implementation Kasra version

kasra fallah

September 2026

1 H_∞ Controller Implementation Guide

This document specifies how to implement the H_∞ controller in our pipeline. The implementation should remain fully self-contained within this file and should consist of two main components:

1. **Offline synthesis:** construct the finite-horizon H_∞ feedback controller from the identified representation dynamics.
2. **Online control:** use the synthesized gains during transformer inference to compute the control action and the final activation intervention at each layer.

The controller should follow the finite-horizon representation-dynamics model

$$x_{k+1} = A_k x_k + B_k u_k + D_k w_k,$$

where:

- x_k is the identified or reduced representation state,
- u_k is the steering control,
- w_k is the normalized adversarial disturbance,
- A_k is the identified layer-to-layer representation dynamics,
- B_k is the steering or control channel,
- D_k is the disturbance channel obtained from the identification procedure.

The corresponding finite-horizon H_∞ objective is

$$\min_{\{u_k\}} \max_{\{w_k\}} \left[x_T^\top Q_f x_T + \sum_{k=0}^{T-1} \left(x_k^\top Q_k x_k + u_k^\top R_k u_k - \gamma^2 w_k^\top w_k \right) \right].$$

The resulting controller has the form

$$u_k = K_k x_k.$$

1.1 Inputs Expected by the Controller

The H_∞ synthesis routine should receive the complete identified system

$$\{A_k, B_k, D_k, Q_k, R_k\}_{k=0}^{T-1}, \qquad Q_f.$$

The H_∞ module should not perform representation-dynamics identification itself. Instead, it should assume that the upstream identification pipeline has already produced

```
A[k] : state dynamics
B[k] : control channel
D[k] : disturbance channel
Q[k] : state or semantic cost
R[k] : intervention cost
Qf   : terminal cost
```

The controller may additionally receive numerical parameters such as

```
gamma_lower
gamma_upper
gamma_tolerance
max_bisection_iterations
numerical_tolerance
```

These quantities are used only during H_∞ synthesis.

1.2 Offline Synthesis

1.2.1 Step 1: Implement a Fixed-γ Solver

Before searching for $\gamma^\star$, first implement a routine that solves the finite-horizon H_∞ problem for one candidate value of γ.

Conceptually, this routine should return

```
solve_for_gamma(gamma)
    -> feasible
    -> gains K
    -> Riccati/value matrices
    -> numerical diagnostics
```

The purpose of this routine is to answer the question:

> For the supplied candidate attenuation level γ, is the finite-horizon H_∞ problem feasible, and if so, what are the corresponding feedback gains?

The $\gamma^\star$ search should only be implemented after this primitive is working correctly.

1.2.2 Step 2: Initialize the Terminal Value Matrix

The backward recursion begins with

$$S_T = Q_f.$$

The matrix S_k plays the same role as the finite-horizon Riccati value matrix in LQR.

The recursion proceeds backward through transformer depth:

$$k = T-1, T-2, \ldots, 0.$$

1.2.3 Step 3: Check Disturbance Feasibility

At layer k, assume that S_{k+1} has already been computed.

Construct

$$M_k = \gamma^2 I - D_k^\top S_{k+1} D_k.$$

The finite-horizon H_∞ recursion is well-defined only when

$$M_k \succ 0.$$

Numerically, this condition should be checked using

$$\lambda_{\min}(M_k) > \varepsilon_{\mathrm{num}},$$

where $\varepsilon_{\mathrm{num}}$ is a small numerical tolerance.

If the condition fails at any layer, the candidate γ should immediately be declared infeasible.

The implementation should record

$$\mathrm{margin}_D = \min_k \lambda_{\min}(M_k),$$

which provides a useful diagnostic indicating how close the synthesis is to the disturbance-feasibility boundary.

1.2.4 Step 4: Robustify the Future Value Matrix

If M_k is positive definite, eliminate the worst-case disturbance analytically and construct

$$\bar{S}_{k+1} = S_{k+1} + S_{k+1} D_k M_k^{-1} D_k^\top S_{k+1}.$$

This is the central modification relative to the nominal LQR recursion.

Conceptually,

$$S_{k+1} \longrightarrow \bar{S}_{k+1}$$

means that the future value function is modified to account for the worst disturbance that can exploit the identified uncertainty directions.

For numerical stability, the implementation should avoid explicitly computing M_k^{-1}. Instead, use a linear solve. For example, the quantity

$$M_k^{-1} D_k^\top S_{k+1}$$

should be obtained by solving

$$M_k X = D_k^\top S_{k+1}.$$

1.2.5 Step 5: Construct the Control Hessian

Using $\bar{S}_{k+1}$, form

$$H_k = R_k + B_k^\top \bar{S}_{k+1} B_k.$$

The controller minimization is well-defined only when

$$H_k \succ 0.$$

Numerically, verify

$$\lambda_{\min}(H_k) > \varepsilon_{\text{num}}.$$

The implementation should record the minimum control margin

$$\text{margin}_U = \min_k \lambda_{\min}(H_k).$$

1.2.6 Step 6: Compute the Feedback Gain

Define

$$F_k = B_k^\top \bar{S}_{k+1} A_k.$$

The H_∞ feedback gain is then

$$K_k = -H_k^{-1} F_k.$$

Equivalently,

$$K_k = -\left(R_k + B_k^\top \bar{S}_{k+1} B_k\right)^{-1} B_k^\top \bar{S}_{k+1} A_k.$$

The sign convention above assumes that online control is written as

$$u_k = K_k x_k.$$

If the runtime API instead uses a tracking error

$$e_k = x_k^{\text{target}} - x_k,$$

then the sign convention must be made consistent throughout the implementation.

As before, K_k should be computed using a linear solve rather than an explicit inverse.

1.2.7 Step 7: Update the Value Matrix

After computing K_k, update

$$S_k = Q_k + A_k^\top \bar{S}_{k+1} A_k - F_k^\top H_k^{-1} F_k.$$

Equivalently,

$$S_k = Q_k + A_k^\top \bar{S}_{k+1} A_k - A_k^\top \bar{S}_{k+1} B_k \left(R_k + B_k^\top \bar{S}_{k+1} B_k\right)^{-1} B_k^\top \bar{S}_{k+1} A_k.$$

After the update, explicitly symmetrize

$$S_k \leftarrow \frac{1}{2}\left(S_k + S_k^\top\right).$$

This avoids accumulation of small floating-point asymmetries during the backward recursion.

1.2.8 Step 8: Repeat Through All Layers

The complete fixed-γ recursion is therefore:

```
S[T] = Qf

for k = T-1 down to 0:

    M_k = gamma^2 I - D[k]^T S[k+1] D[k]

    check M_k > 0

    S_bar =
        S[k+1]
        + S[k+1] D[k]
          M_k^{-1}
          D[k]^T S[k+1]

    H_k =
        R[k]
        + B[k]^T S_bar B[k]

    check H_k > 0

    F_k =
        B[k]^T S_bar A[k]

    K[k] =
        - H_k^{-1} F_k

    S[k] =
        Q[k]
        + A[k]^T S_bar A[k]
        - F_k^T H_k^{-1} F_k

    S[k] = 0.5 * (S[k] + S[k]^T)
```

If every layer succeeds, the candidate γ is feasible.

1.3 Searching for $\gamma^\star$

1.3.1 Step 9: Define the Robust Attenuation Boundary

The quantity required by the paper is

$$\gamma^\star = \inf \{\gamma : the finite - horizon H$$

$_\infty$ recursion is feasible .

This quantity should be obtained using feasibility search rather than gradient-based optimization.

1.3.2 Step 10: Find a Feasible Upper Bound

Begin with an initial value γ_{upper} and run the fixed-γ solver.

If the recursion is infeasible, increase the attenuation level, for example,

$$\gamma_{\text{upper}} \leftarrow 2\gamma_{\text{upper}},$$

and test again.

Continue until either:

1. a feasible γ_{upper} is found, or
2. a specified maximum search value is reached.

If no feasible value is found, the synthesis routine should return

```
feasible = False
gamma_star = None
```

The controller should not silently fall back to LQR.

1.3.3 Step 11: Perform Bisection

Once a feasible upper bound and an infeasible lower bound are available, use bisection.

At each iteration,

$$\gamma_{\text{mid}} = \frac{\gamma_{\text{low}} + \gamma_{\text{high}}}{2}.$$

Run the fixed-γ solver.

If γ_{mid} is feasible, update

$$\gamma_{\text{high}} = \gamma_{\text{mid}}.$$

Otherwise update

$$\gamma_{\text{low}} = \gamma_{\text{mid}}.$$

Continue until

$$\gamma_{\text{high}} - \gamma_{\text{low}} < \varepsilon_\gamma,$$

or until the maximum number of bisection iterations is reached.

The final estimate is

$$\gamma^\star \approx \gamma_{\text{high}},$$

where the upper side is used because it is known to remain feasible.

After estimating $\gamma^\star$, rerun the fixed-γ solver at that value and retain the resulting gains as the final controller.

1.3.4 Optional Numerical Safety Margin

Because operation exactly at the feasibility boundary can be numerically fragile, deployment may use

$$\gamma_{\text{deploy}} = (1+\epsilon)\gamma^\star,$$

with a small value such as

$$\epsilon \in [10^{-3}, 10^{-2}].$$

If this is used, the implementation should distinguish between

$$\gamma^\star$$

and

$$\gamma_{\text{used}}.$$

The quantity reported in the paper should remain $\gamma^\star$.

1.4 Offline Outputs

The offline synthesis routine should return four groups of quantities.

1.4.1 Feedback Gains

Store the complete layer-wise controller

$$\{K_k\}_{k=0}^{T-1}.$$

These matrices are the only objects required for online H_∞ feedback.

1.4.2 Robust Steerability

Store

$$\gamma^\star.$$

The robust-steerability score used in the paper is

$$\mathcal{S}_{\text{robust}} = \frac{1}{\gamma^\star}.$$

1.4.3 Feasibility Flag

The result should expose an explicit Boolean feasibility result.

For example,

```
feasible = True / False
```

Feasibility should not be inferred indirectly from the existence of gain matrices.

1.4.4 Numerical Diagnostics

At minimum, store

```
gamma_star
gamma_used
gamma_lower_final
gamma_upper_final
bisection_iterations
bisection_converged

min_disturbance_margin
min_control_margin

max_condition_number_M
max_condition_number_H

max_norm_S
max_norm_K

failed_layer
failure_reason
```

The fields `failed_layer` and `failure_reason` are especially useful for diagnosing synthesis failures.

1.5 Online Control

1.5.1 Step 12: Apply the Synthesized Gain

No optimization should be performed online.

At transformer layer k, retrieve the previously synthesized feedback gain K_k.

Given the current controller state x_k, compute

$$u_k = K_k x_k.$$

The adversarial disturbance maximization is not solved during inference. Its effect has already been incorporated into K_k during offline synthesis.

1.5.2 Step 13: Map the Control to the Activation Intervention

The control variable u_k may live in a lower-dimensional control coordinate system.

The actual activation-space intervention is therefore

$$\Delta h_k = B_k u_k.$$

The online controller should conceptually perform

```
current representation
        |
        v
construct controller state x_k
        |
        v
u_k = K_k x_k
        |
        v
delta_h_k = B_k u_k
        |
        v
inject delta_h_k into transformer activation
```

If the control channel is directly additive,

$$B_k = I,$$

then

$$\Delta h_k = u_k.$$

1.5.3 Step 14: Controller State

For the finite-horizon state-feedback formulation

$$u_k = K_k x_k,$$

there is no additional dynamic controller state.

In particular, H_∞ does not require PID-like quantities such as:

- accumulated error,
- previous error,
- derivative state,
- integral controller state.

All controller memory is contained in the precomputed layer-dependent gain sequence $\{K_k\}$.

1.5.4 Step 15: Reset Logic

Because the controller is stateless, the mathematical reset operation is a no-op.

The implementation may reset lightweight runtime bookkeeping such as

```
current_layer = 0
```

if the class internally tracks layer progression.

No H_∞ state needs to be reset.

1.6 Connection to the Identification Procedure

1.6.1 Step 16: Interpretation of D_k

The disturbance matrix should encode uncertainty measured by the representation-dynamics identification pipeline.

Suppose the nominal identified dynamics are

$$x_{k+1} \approx A_k x_k + B_k u_k.$$

For calibration trajectory i, define the residual

$$\xi_k^{(i)} = x_{k+1}^{(i)} - A_k x_k^{(i)} - B_k u_k^{(i)}.$$

Estimate the layer-wise residual covariance

$$\Sigma_{\xi,k} = Cov\left(\xi_k\right).$$

The disturbance matrix should satisfy approximately

$$D_k D_k^\top \approx \Sigma_{\xi,k}.$$

For example, if

$$\Sigma_{\xi,k} = U_k \Lambda_k U_k^\top,$$

then one natural choice is

$$D_k = U_k \Lambda_k^{1/2}.$$

Under this construction, w_k is a normalized disturbance while D_k determines the directions and magnitudes of uncertainty observed during calibration.

This is generally preferable to setting

$$D_k = I,$$

because $\gamma^\star$ then reflects the empirically identified uncertainty geometry.

1.6.2 Step 17: Topic Distribution Shift Interpretation

Suppose the nominal dynamics identified on the calibration-topic distribution are A_k, while an out-of-distribution topic induces effective dynamics

$$A_k^{\mathrm{OOD}} = A_k + \Delta A_k.$$

Then

$$x_{k+1} = A_k x_k + B_k u_k + \underbrace{\Delta A_k x_k}_{effectivedynamicaldisturbance}.$$

Therefore the mismatch generated by topic distribution shift naturally enters the H_∞ model through the disturbance channel.

The conceptual pipeline is

$$identificationuncertainty \rightarrow D_k \rightarrow H$$

$_\infty$ synthesis $\rightarrow \gamma^\star \rightarrow robuststeerability.$

1.7 Numerical Validation

1.7.1 Large-γ Recovery of LQR

The most important implementation check is the limit

$$\gamma \to \infty.$$

Since

$$\left(\gamma^2 I - D_k^\top S_{k+1} D_k\right)^{-1} \to 0,$$

we obtain

$$\bar{S}_{k+1} \to S_{k+1}.$$

Therefore,

$$K_k^{H_\infty} \to K_k^{\mathrm{LQR}}.$$

Before any language-model experiment is run, the implementation should verify numerically that sufficiently large γ reproduces the LQR gain sequence.

1.7.2 Zero-Disturbance Recovery of LQR

Setting

$$D_k = 0$$

should immediately give

$$\bar{S}_{k+1} = S_{k+1},$$

and therefore

$$K_k^{H_\infty} = K_k^{\mathrm{LQR}}.$$

This provides a second simple unit test.

1.7.3 Monotonic Feasibility

The $\gamma^\star$ search assumes monotonic feasibility.

If γ_1 is feasible and

$$\gamma_2 > \gamma_1,$$

then γ_2 should also be feasible.

Thus,

$$\gamma_1 feasible \quad \Longrightarrow \quad \gamma_2 > \gamma_1 feasible.$$

A strong violation of this property generally indicates a numerical problem or an incorrect implementation.

1.8 Recommended Implementation Order

The recommended sequence for implementing `h_infinity.py` is:

1. Implement input and dimension validation.
2. Implement the fixed-γ backward recursion.
3. Validate the fixed-γ implementation using very large γ and compare the gains against the existing LQR implementation.
4. Add the disturbance-feasibility check
$$\gamma^2 I - D_k^\top S_{k+1} D_k \succ 0.$$
5. Add numerical diagnostics for M_k, H_k, S_k, and K_k.
6. Implement automatic search for an initial feasible upper bound on γ.
7. Implement bisection to estimate $\gamma^\star$.
8. Rerun synthesis at the final feasible value and store the final gain sequence.
9. Implement online feedback
$$u_k = K_k x_k.$$
10. Implement the activation intervention
$$\Delta h_k = B_k u_k.$$
11. Implement `reset()` as stateless or bookkeeping-only reset logic.
12. Validate
$$D_k = 0 \quad \Longrightarrow \quad H_\infty = LQR,$$
and
$$\gamma \gg 1 \quad \Longrightarrow \quad H_\infty \approx LQR.$$
13. Only after the above checks succeed, integrate the controller into transformer inference experiments.